

MINI PROJECT REPORT 2026

Real-Time Indian Stock and Crypto Portfolio Management System
with AI-Powered Analytics, Live Market Data, and Aggregated
Financial News

PROJECT TYPE	Full-Stack Web Application
TECHNOLOGY STACK	Python Flask + SQLite + Vanilla JS
FRONTEND	HTML5, CSS3, Chart.js, GSAP
EXTERNAL APIS	Yahoo Finance, Google Gemini, Google OAuth

Table of Contents

1. Project Synopsis (Project Proposal)

2. Requirement Specification (Software and Hardware)

3. Feasibility Study

A. Technical Feasibility

B. Economical Feasibility

C. Organizational Feasibility

D. Cultural Feasibility

4. Event Table

5. ER Diagram (Entity-Relationship)

6. System Design Details

A. Use Case Diagram

B. Class Diagram

C. Activity Diagrams

D. Sequence Diagrams

E. Component Diagram

F. Deployment Diagram

7. API Endpoint Reference

8. File Structure

Section 1

Project Synopsis (Project Proposal)

A comprehensive overview of the project objectives, problem statement, proposed solution, and scope.

1.1 Problem Statement

Individual investors in India lack a unified, free, and intelligent platform to track stock and cryptocurrency holdings in real-time. Existing platforms like Groww and Zerodha are broker-locked -- they only show holdings purchased through their own service. An investor with accounts across multiple brokers has no single dashboard to see their total net worth, overall profit/loss, and allocation breakdown.

1.2 Proposed Solution

A full-stack web application that allows users to:

- **Register/Login** securely (email+password or Google OAuth 2.0).
- **Add holdings** from any Indian stock exchange (NSE/BSE) or cryptocurrency.
- **Track live prices** via Yahoo Finance API integration.
- **View analytics** -- allocation pie charts, sector distribution, and performance metrics.
- **Ask AI** -- an AI-powered portfolio analyst (Gemini/Grok) that answers questions about diversification, risk, and performance.
- **Read curated news** -- aggregated from Moneycontrol RSS and Yahoo Finance, matched to the user's specific holdings.

1.3 Scope

IN SCOPE	OUT OF SCOPE
Multi-user authentication (email + Google OAuth)	Actual trade execution (buy/sell orders)
CRUD operations on holdings	Broker API integration (Zerodha/Groww)
Live price fetching via Yahoo Finance	Tax computation or ITR filing
AI-powered portfolio analysis	Mobile native app (Android/iOS)
News aggregation matched to holdings	Real-time WebSocket price streaming
Admin dashboard for owner oversight	Multi-currency portfolios

1.4 Objectives

- 1. Build a responsive, premium-quality single-page web application.
- 2. Implement secure authentication with PBKDF2 password hashing and Google OAuth 2.0.
- 3. Provide real-time market data via a CORS-bypass proxy to Yahoo Finance.
- 4. Deliver intelligent portfolio insights using AI (Gemini API) with a local fallback analyzer.
- 5. Aggregate and display portfolio-relevant news from multiple sources.

1.5 Technology Stack -- Justification

LAYER	TECHNOLOGY	WHY WE USED IT	WHAT IF WE DON'T USE IT
Backend	Python Flask	Lightweight, minimal boilerplate, perfect for REST APIs. Single-file server deployment.	Express.js (Node) or Django -- both heavier. Django has ORM overhead; Express lacks built-in session management.
Database	SQLite	Zero-configuration, file-based, serverless. No separate DB server needed.	PostgreSQL/MySQL require separate server setup, hosting costs, connection pooling -- overkill for a mini project.
Frontend	Vanilla HTML/CSS/JS	Full control, no build step, no framework lock-in. Loads instantly.	React/Vue add npm build pipelines, 200+ node_modules packages, slower initial load.
Charts	Chart.js	Free, lightweight, responsive canvas-based charts with excellent documentation.	D3.js is far more complex. Highcharts requires a commercial license.
Animations	GSAP	Industry-standard, hardware-accelerated, smooth ScrollTrigger integration.	CSS-only animations lack scroll-triggered reveals and complex sequencing.
Market Data	Yahoo Finance API	Free, no API key required, covers NSE/BSE/Crypto with real-time prices.	Alpha Vantage: 5 calls/min limit. NSE official API blocks scrapers.
AI	Gemini + Grok APIs	Free tier available, multi-model fallback ensures reliability.	Without AI, users only get static numbers -- no personalized insights.
News	Moneycontrol RSS + Yahoo News	Free, no authentication. Moneycontrol covers Indian markets specifically.	Paid news APIs cost \$50-200/month.
Auth	PBKDF2 + Google OAuth 2.0	PBKDF2 is NIST-recommended. OAuth eliminates password management for users.	Plain-text passwords are a critical security vulnerability.
Session	Flask signed cookies	Stateless, no server-side session store. Cryptographically signed (HMAC).	Server-side sessions (Redis) add infrastructure complexity.

Section 2

Requirement Specification

Software, hardware, functional, and non-functional requirements.

2.1 Software Requirements

CATEGORY	REQUIREMENT	VERSION
Operating System	Windows 10/11, macOS, or Linux	64-bit
Runtime	Python	3.8+
Web Framework	Flask	2.x+
Database	SQLite3	Built into Python
Browser	Chrome / Edge / Firefox	Latest
Package Manager	pip	Bundled

Note: Flask is the **only** external Python dependency. All other modules (sqlite3, hashlib, json, urllib, socket, os, xml, re, concurrent.futures, uuid) are part of the Python standard library.

2.2 Hardware Requirements

COMPONENT	MINIMUM	RECOMMENDED
Processor	Intel i3 / AMD Ryzen 3	Intel i5 / AMD Ryzen 5+
RAM	4 GB	8 GB+
Storage	500 MB free	1 GB free
Network	Active internet	10+ Mbps broadband
Display	1366×768	1920×1080 Full HD

2.3 Functional Requirements

ID	REQUIREMENT	PRIORITY
FR-01	User registration with email and password	High
FR-02	User login with email/password or Google OAuth 2.0	High
FR-03	Password reset via security question	Medium
FR-04	Add, edit, and delete stock/crypto holdings	High
FR-05	Live price fetching from Yahoo Finance	High
FR-06	Portfolio summary: net worth, invested, P and L, day change	High
FR-07	Analytics dashboard: allocation charts, sector breakdown	High
FR-08	AI-powered portfolio analyst (Ask AI tab)	Medium
FR-09	Stock news aggregation matched to user holdings	Medium
FR-10	Admin dashboard for owner oversight	Low
FR-11	Session persistence across browser refreshes	High
FR-12	Responsive design for mobile and desktop	High

2.4 Non-Functional Requirements

ID	REQUIREMENT	DETAILS
NFR-01	Security	PBKDF2-HMAC-SHA256 (100K iterations). HttpOnly, SameSite=Lax cookies.
NFR-02	Performance	60-second price refresh. 10-minute news cache. Sub-2s page load.
NFR-03	Scalability	SQLite supports up to 140 TB. Flask handles 100+ concurrent users.
NFR-04	Usability	Premium dark theme, smooth animations, single-page architecture.
NFR-05	Availability	Runs locally. Market data depends on Yahoo Finance availability.
NFR-06	Maintainability	Single app.py backend. Clear REST API routes. Modular JS functions.

Section 3

Feasibility Study

Analysis of whether the project is achievable from four key perspectives.

A. Technical Feasibility FEASIBLE

FACTOR	ASSESSMENT
Language Maturity	Python (30+ years). Flask has 67K+ GitHub stars.
Database	SQLite is embedded in Python -- zero installation. Most deployed database in the world.
API Availability	Yahoo Finance: free, no key. Gemini API: free tier (15 RPM).
CORS Bypass	Flask proxy route forwards requests server-side where CORS does not apply. Without this, all price fetches would be blocked by the browser.
OAuth 2.0	Google OAuth is well-documented. Flask session handles CSRF state tokens.
Browser Support	ES6+ JavaScript supported since 2017 in all modern browsers.

Why not Django? Django includes ORM, admin panel, template engine, migration system. For 2 tables and a single-page app, this overhead is unnecessary. Flask gives full control with minimal abstraction.

B. Economical Feasibility

FEASIBLE -- Zero Cost

COST CATEGORY	AMOUNT	JUSTIFICATION
Software Licenses	Rs. 0	All tools are free and open source
Market Data API	Rs. 0	Yahoo Finance requires no API key
AI API	Rs. 0	Gemini free tier: 15 RPM, 1500/day
Hosting + Domain + SSL	Rs. 0	Runs on localhost
Database	Rs. 0	SQLite is serverless and free
Total	Rs. 0	

C. Organizational Feasibility

FEASIBLE

FACTOR	ASSESSMENT
Team Size	Designed for 1-3 developers. Monolithic architecture eliminates coordination overhead.
Deployment	Run python app.py and open browser. No Docker or CI/CD required.
Training	Self-explanatory UI with labeled tabs. No training needed.
Maintenance	Self-contained codebase. Auto migrations in init_db().

D. Cultural Feasibility **FEASIBLE**

FACTOR	ASSESSMENT
Target Audience	Indian retail investors (18-45) familiar with Groww, Zerodha, Angel One.
Language	English -- standard for Indian financial apps. NSE/BSE symbol conventions.
Currency	Indian Rupees with lakhs/crores formatting.
Privacy	Data stored locally (SQLite file). No third-party data sharing except price/AI fetches.
Digital Literacy	800M+ Indian internet users. Stock investors use web apps regularly.

Section 4

Event Table

All system events, their triggers, sources, activities, and responses.

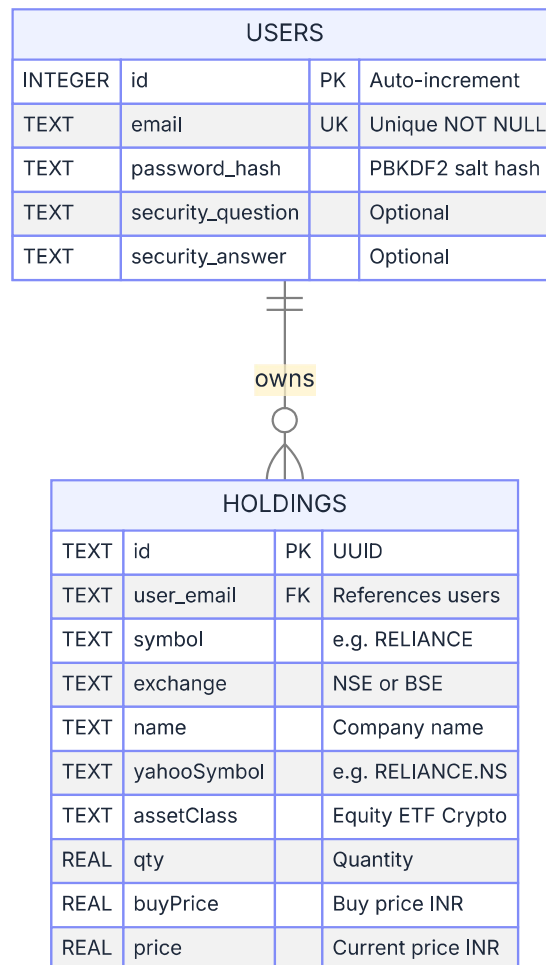
#	EVENT	TRIGGER	SOURCE	ACTIVITY	RESPONSE
E1	Registration	Click Sign Up	User	Validate, hash password (PBKDF2), insert users table, create session	Dashboard loads
E2	Login (Email)	Click Sign In	User	Verify PBKDF2 hash, create session cookie	Dashboard loads
E3	Login (Google)	Click Google button	User	OAuth redirect, exchange code, fetch email, create/find user	Dashboard loads
E4	Add Holding	Submit form	User	Generate UUID, fetch price, POST to /api/holdings	Row appears with live price
E5	Delete Holding	Click delete icon	User	Remove from array, POST updated list, replace in DB	Row removed, totals updated
E6	Price Refresh	60s timer	System	Fetch via /proxy/ from Yahoo Finance, update price field	P and L and net worth updated
E7	View Analytics	Click Analytics tab	User	Calculate allocation %, render Chart.js charts	Pie/bar charts shown
E8	Ask AI	Type + Send	User	POST /api/ask-ai, try Gemini, try Grok, fallback local	Markdown AI response
E9	View News	Click News tab	User	GET /api/news, fetch 4 MC feeds + Yahoo, match symbols	News cards displayed
E10	Logout	Click Logout	User	POST /api/logout, clear session cookie	Login screen shown
E11	Password Reset	Click Forgot	User	Verify security question, update hash in DB	Password updated
E12	Admin Access	Navigate /admin	Admin	Validate secret key, query all data	Admin dashboard

Section 5

ER Diagram (Entity-Relationship)

Database schema showing entities, attributes, and relationships.

ENTITY-RELATIONSHIP DIAGRAM

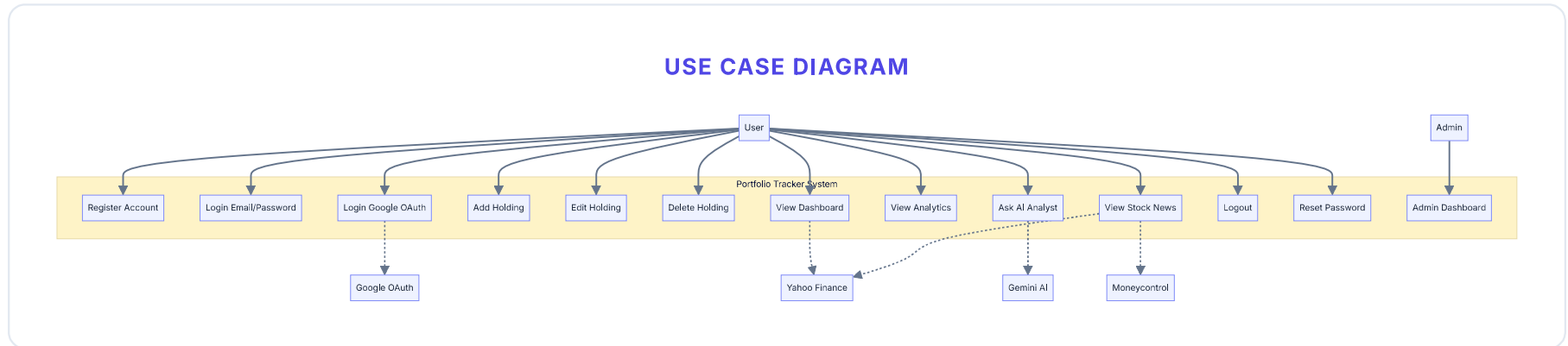


Design Justifications

DECISION	RATIONALE
No prices table	Prices fetched in real-time from Yahoo Finance every 60s. Historical storage would require background schedulers and gigabytes of data.
No sessions table	Flask uses cryptographically signed cookies. Session data stored in cookie itself (HMAC-SHA256 signed).
Composite PK (id, user_email)	Each UUID unique per user. ON DELETE CASCADE auto-removes orphaned holdings.
Separate yahooSymbol column	Yahoo uses different format (RELIANCE.NS) than NSE tickers (RELIANCE). Avoids repeated concatenation.

Section 6A

Use Case Diagram



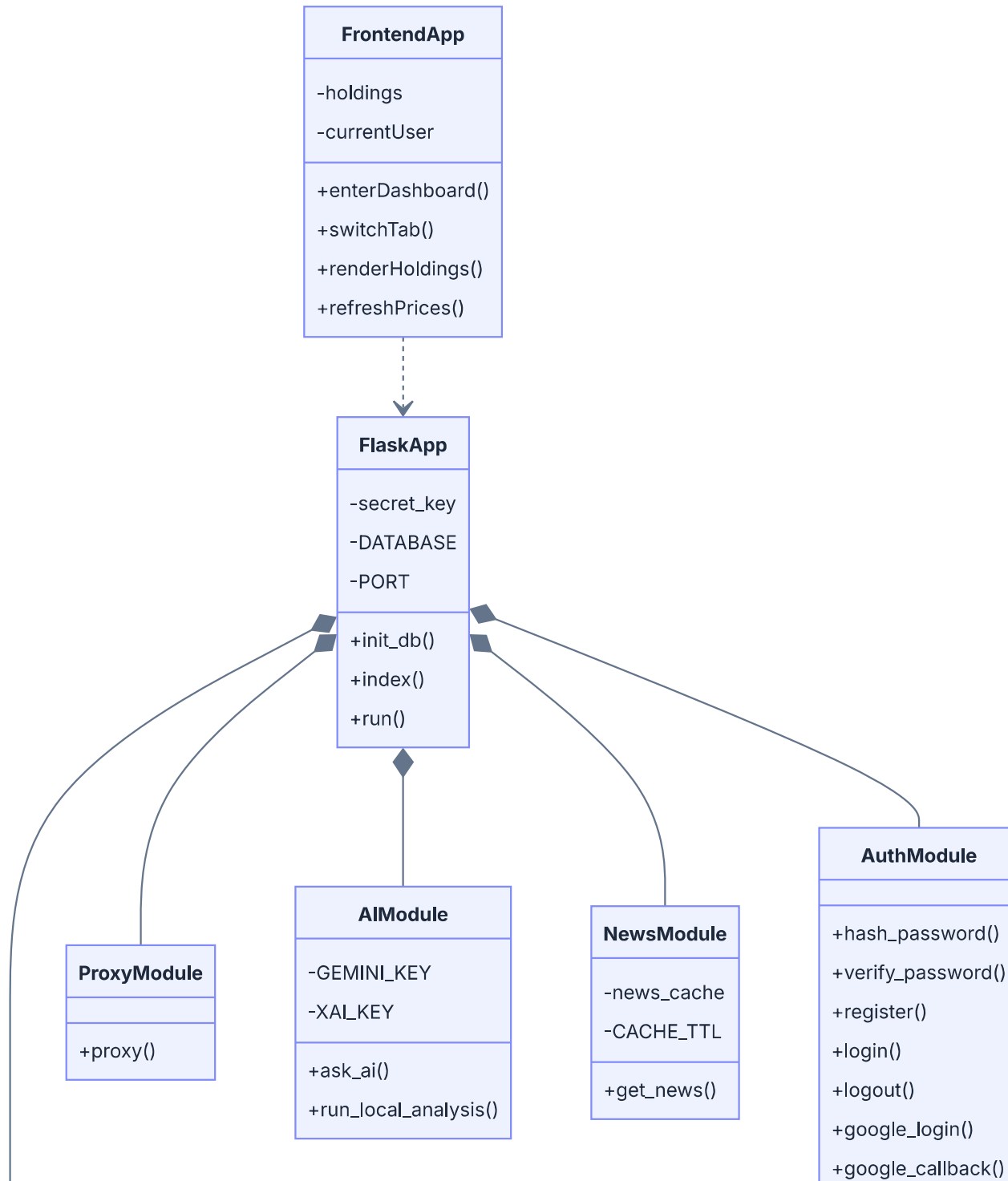
Use Case Descriptions

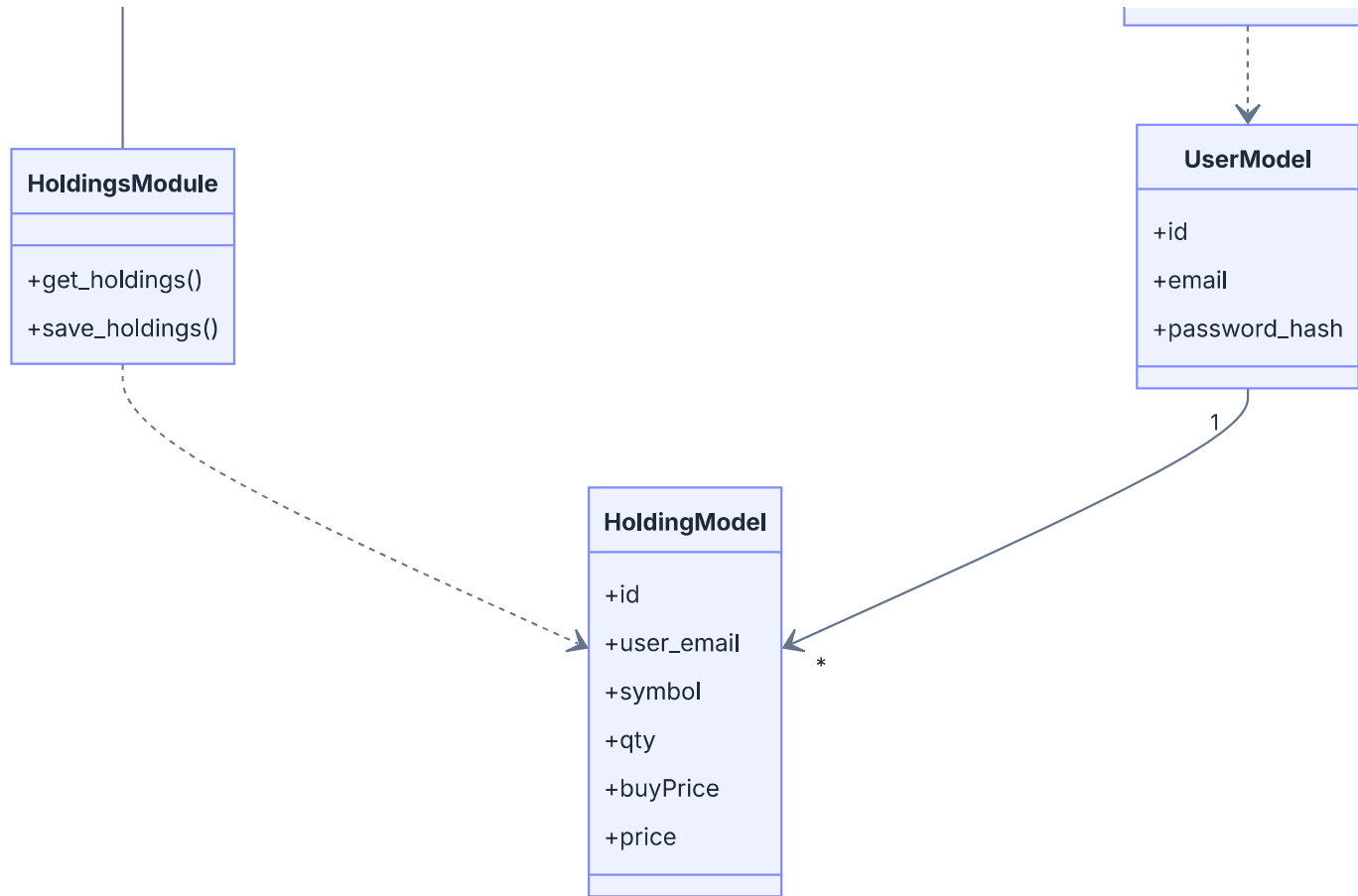
USE CASE	ACTOR	PRECONDITION	POSTCONDITION
Register	User	Not logged in	Account created, session started
Login (Email)	User	Account exists	Session cookie set, dashboard loaded
Login (Google)	User	Has Google account	Auto-created if new, session set
Add Holding	User	Logged in	Holding saved, price fetched
View Dashboard	User	Logged in, has holdings	Net worth, P and L displayed
Ask AI	User	Logged in	AI response in Markdown
Admin Panel	Admin	Knows secret key	All data visible

Section 6B

Class Diagram

CLASS DIAGRAM



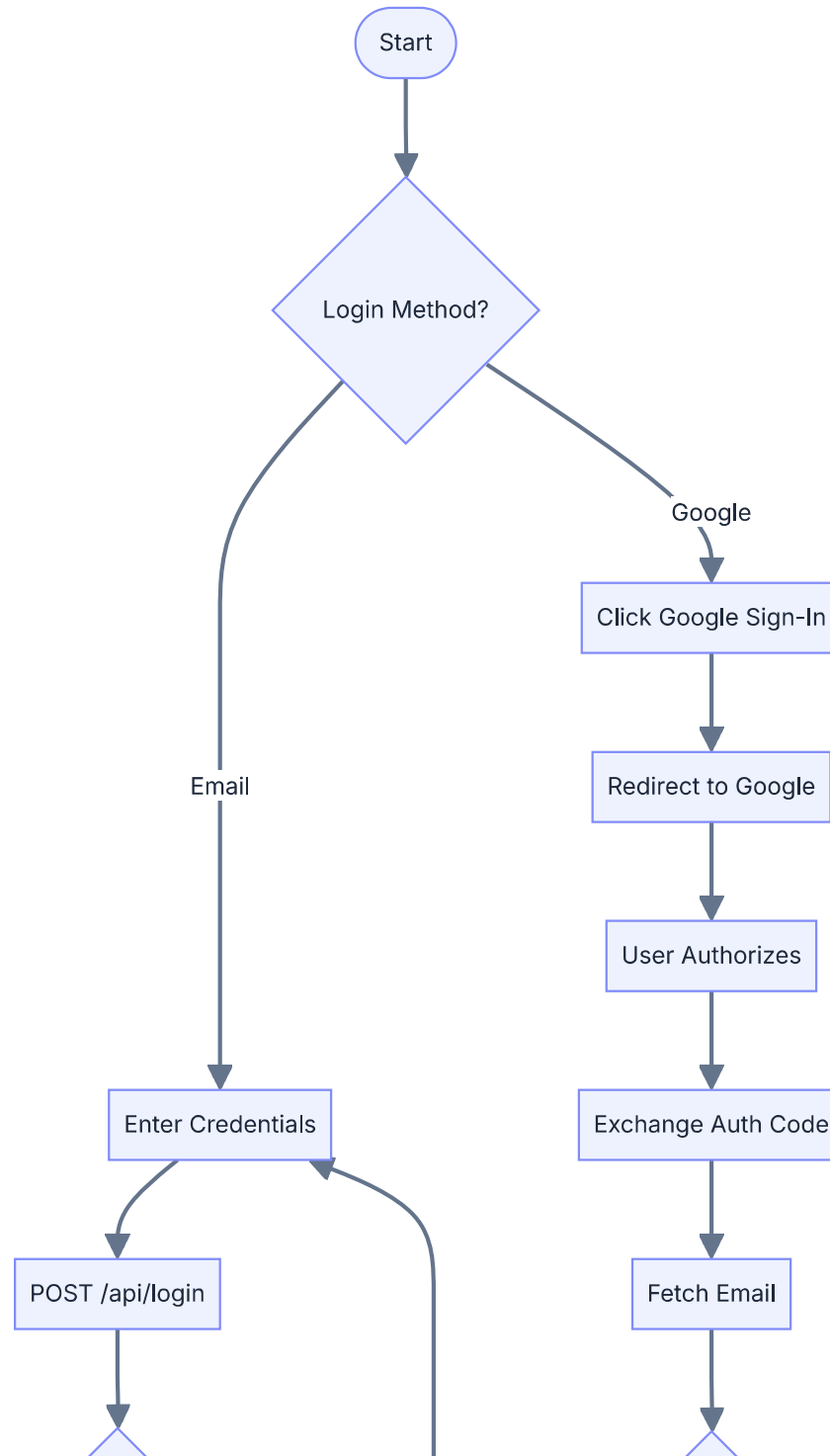


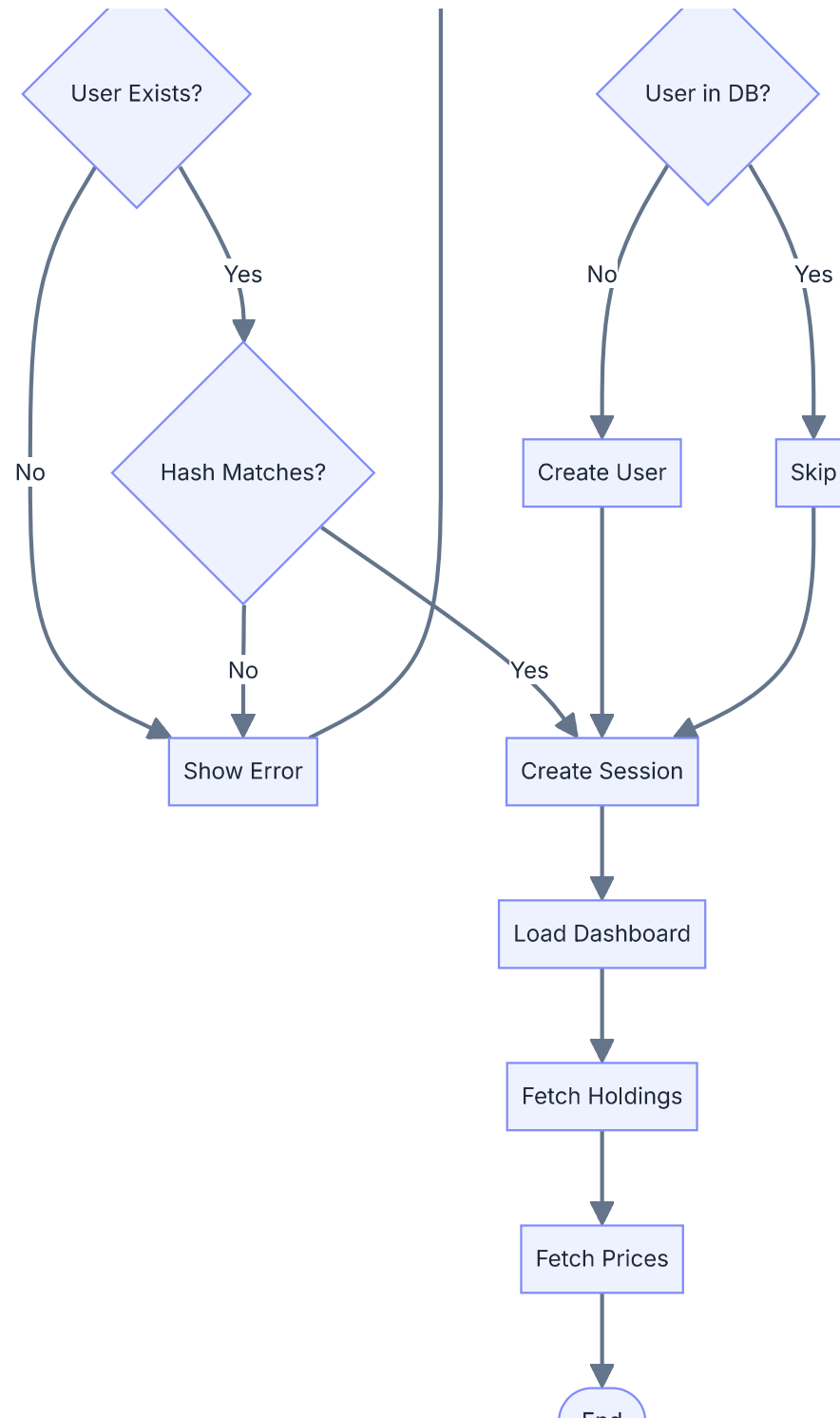
Section 6C

Activity Diagrams

C1. User Login Flow

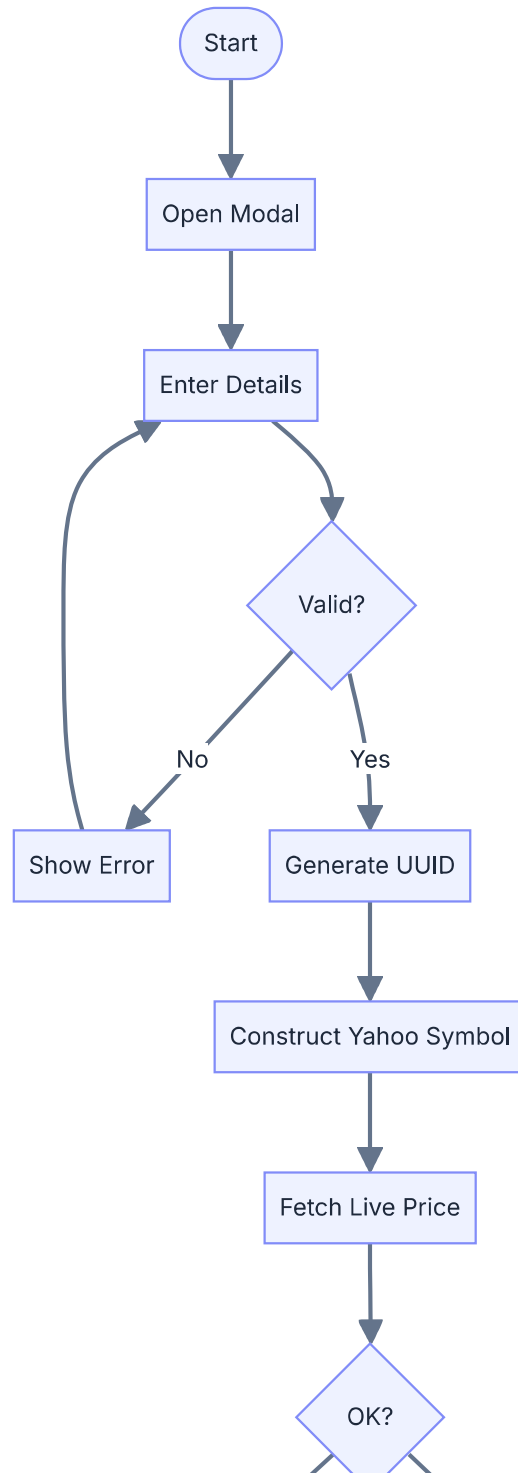
ACTIVITY DIAGRAM -- USER LOGIN

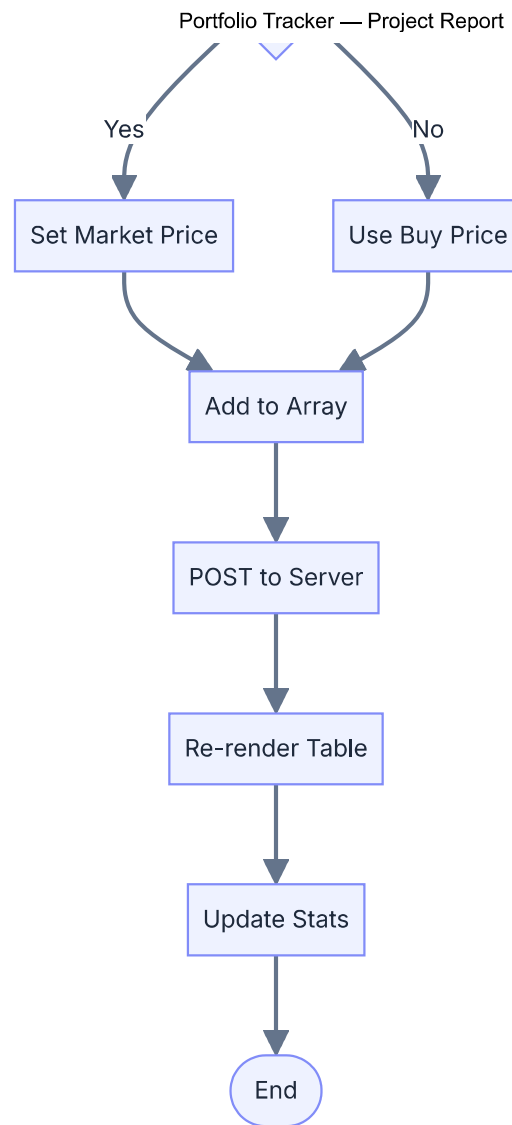




C2. Add Holding Flow

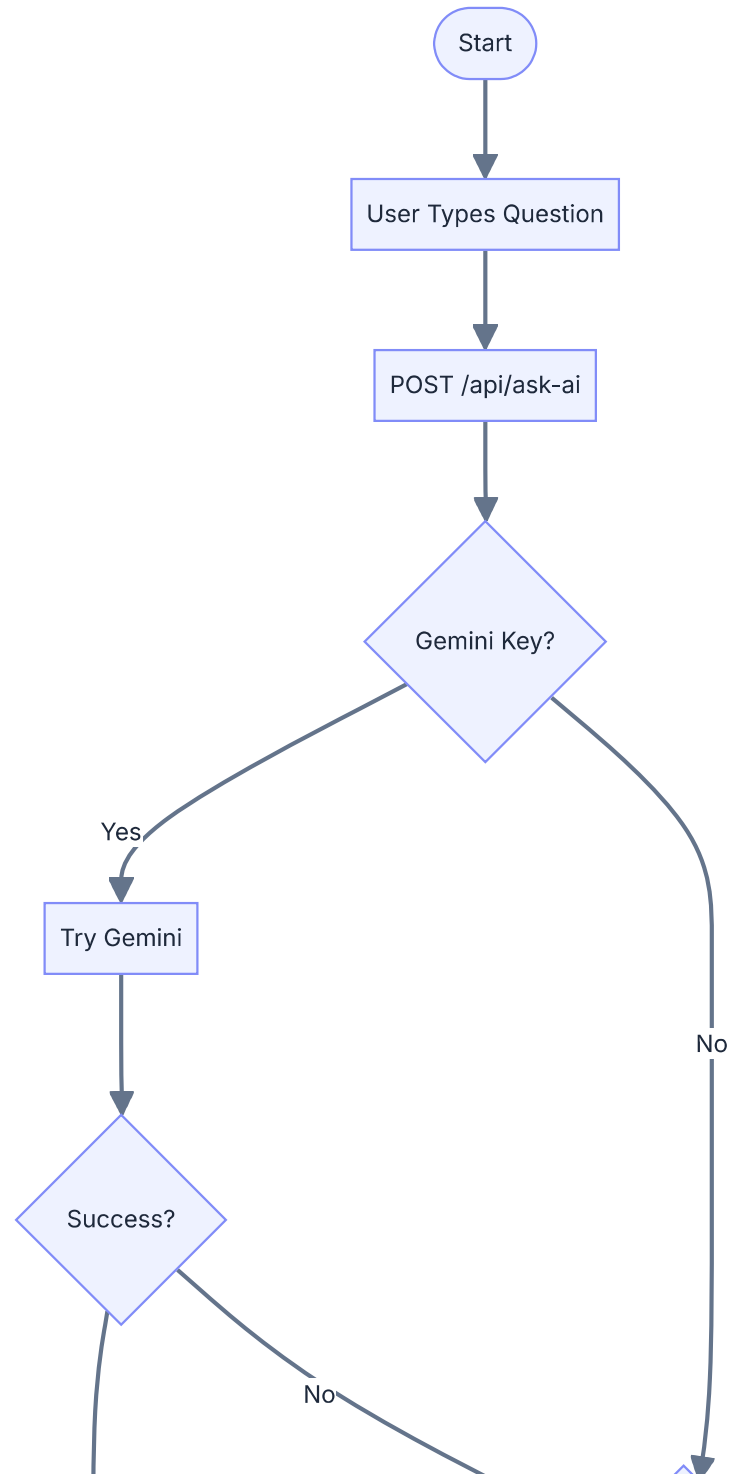
ACTIVITY DIAGRAM -- ADD HOLDING

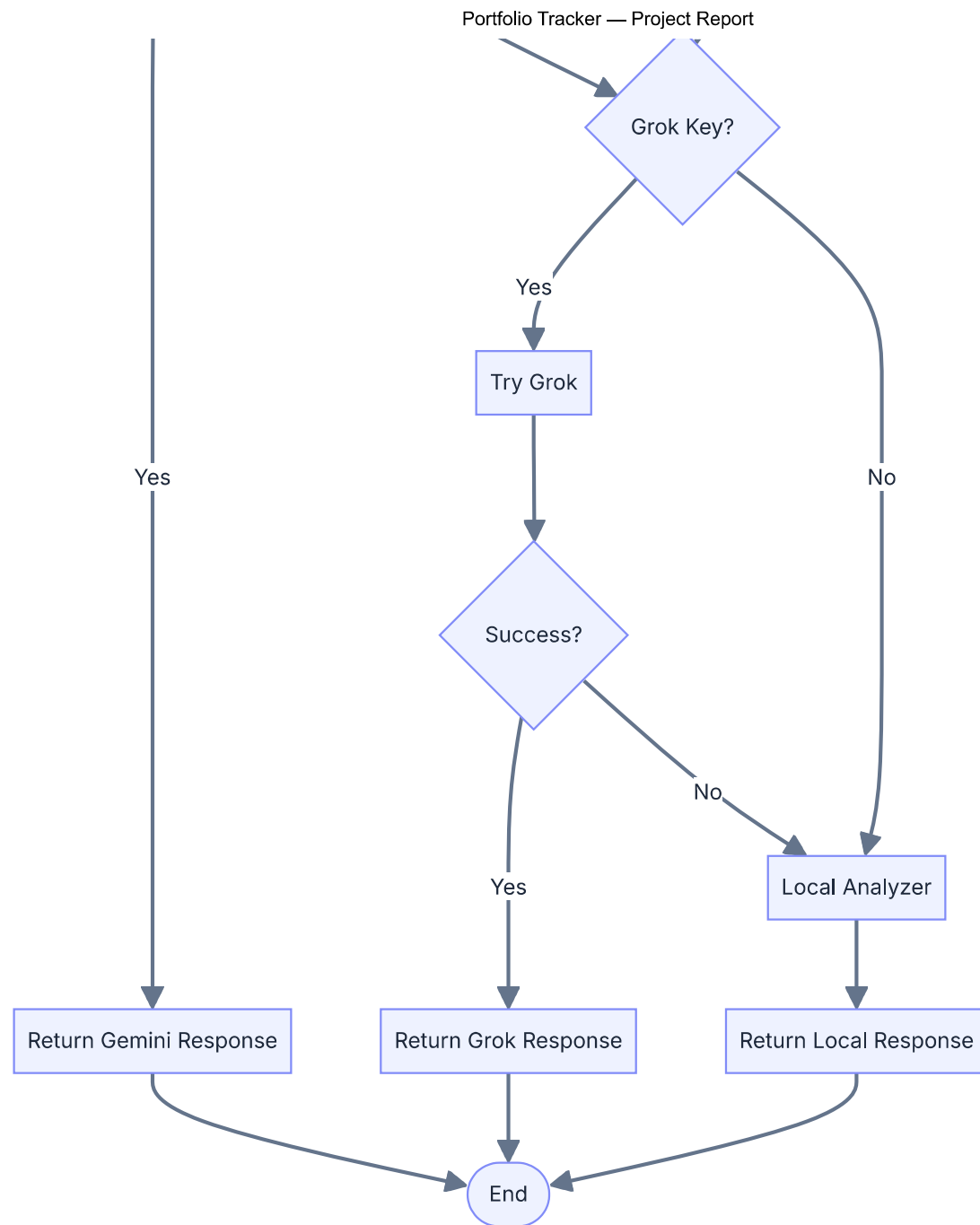




C3. AI Query with Fallback

ACTIVITY DIAGRAM -- AI ANALYSIS



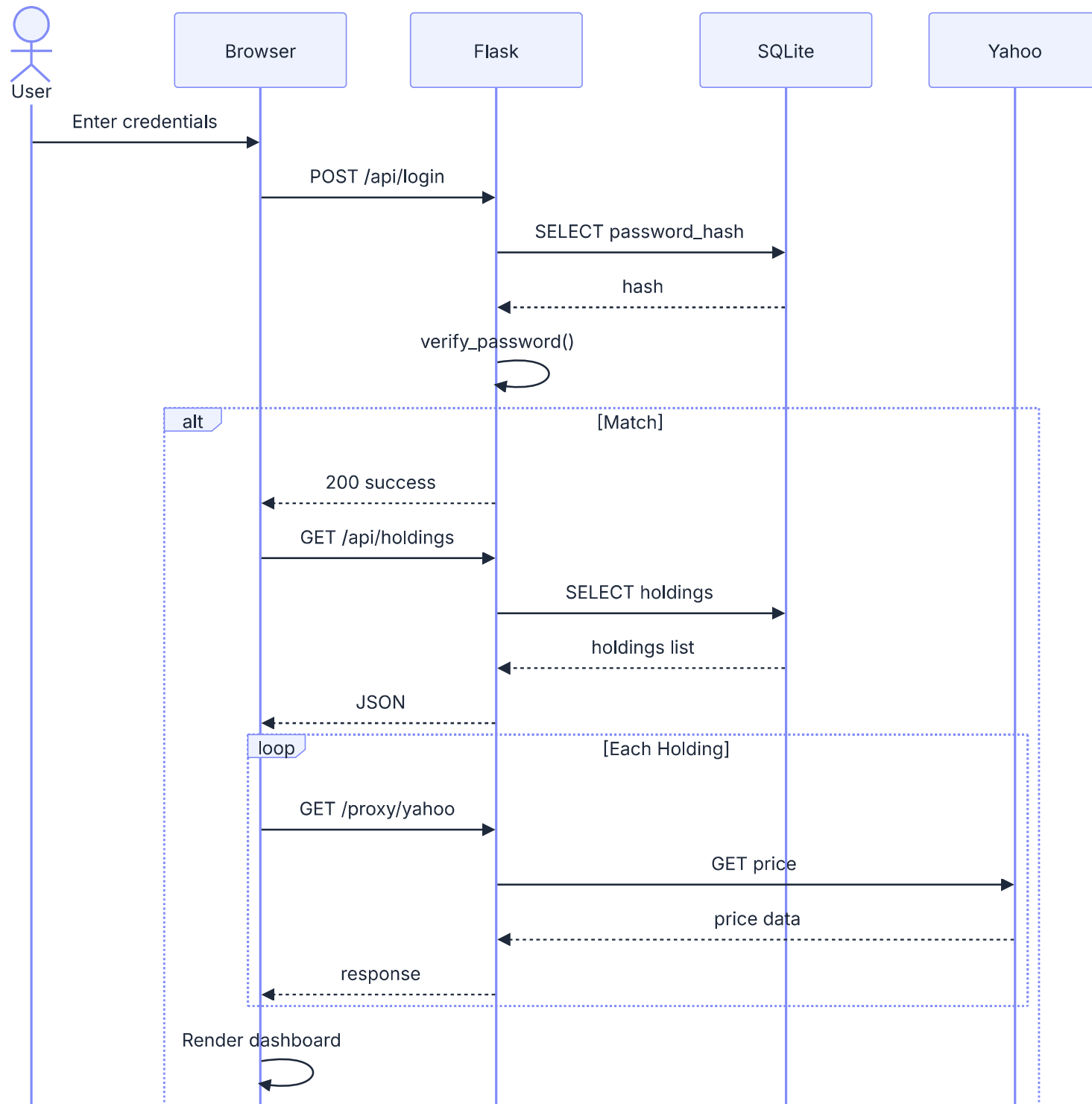


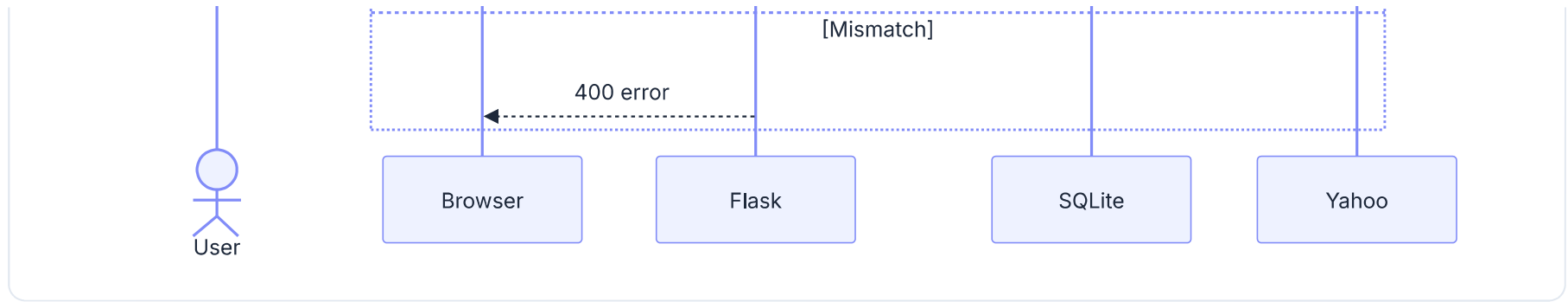
Section 6D

Sequence Diagrams

D1. Login and Dashboard Load

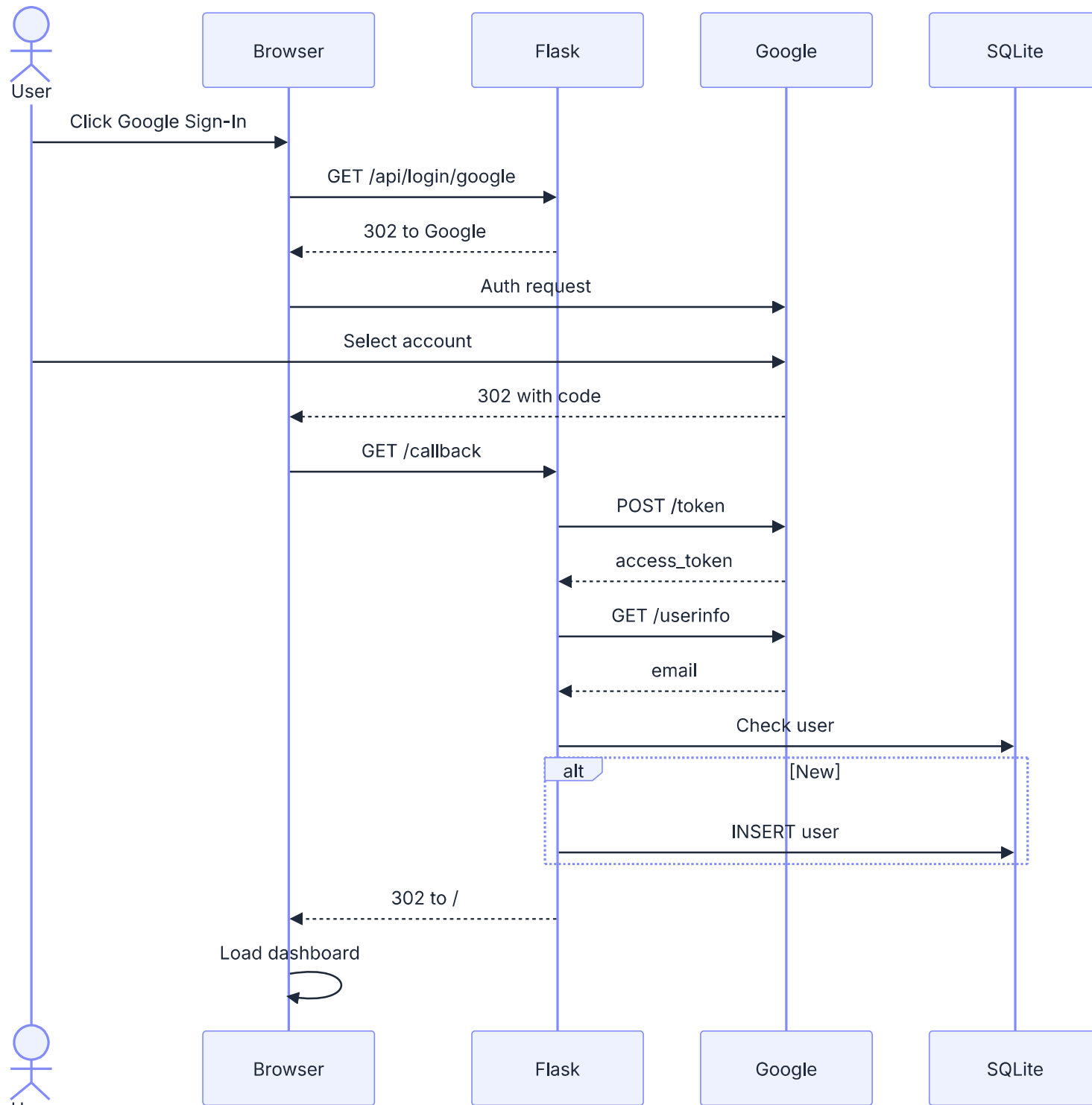
SEQUENCE DIAGRAM -- LOGIN FLOW





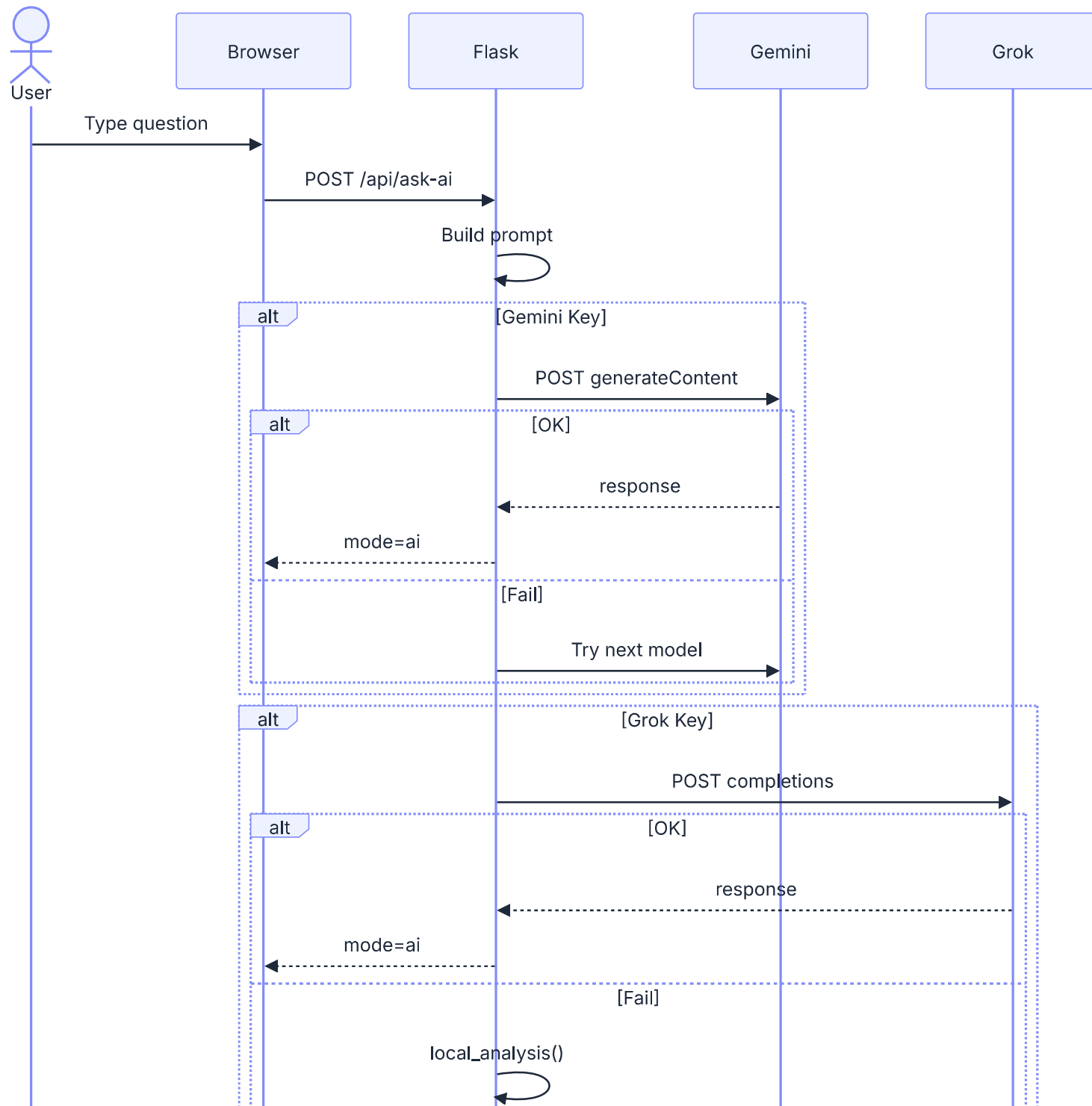
D2. Google OAuth Flow

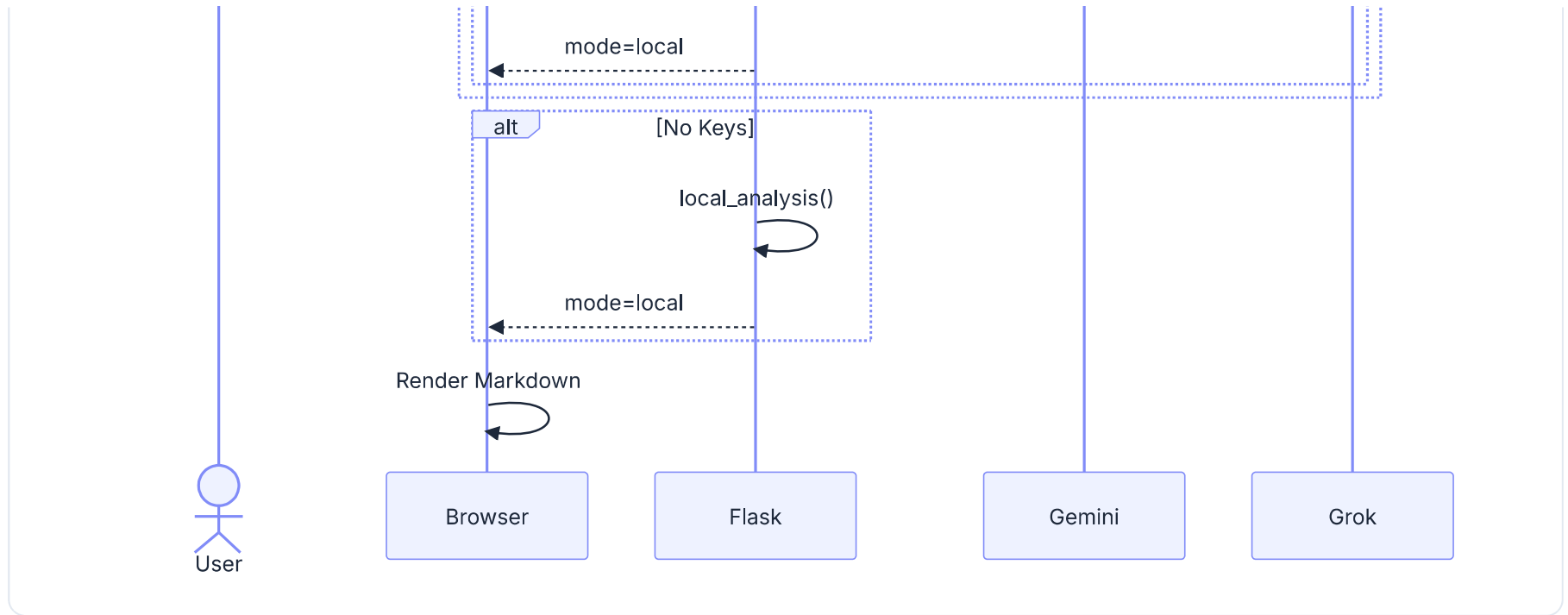
SEQUENCE DIAGRAM -- GOOGLE OAUTH 2.0



D3. AI Query with Fallback

SEQUENCE DIAGRAM -- AI MULTI-PROVIDER FALLBACK

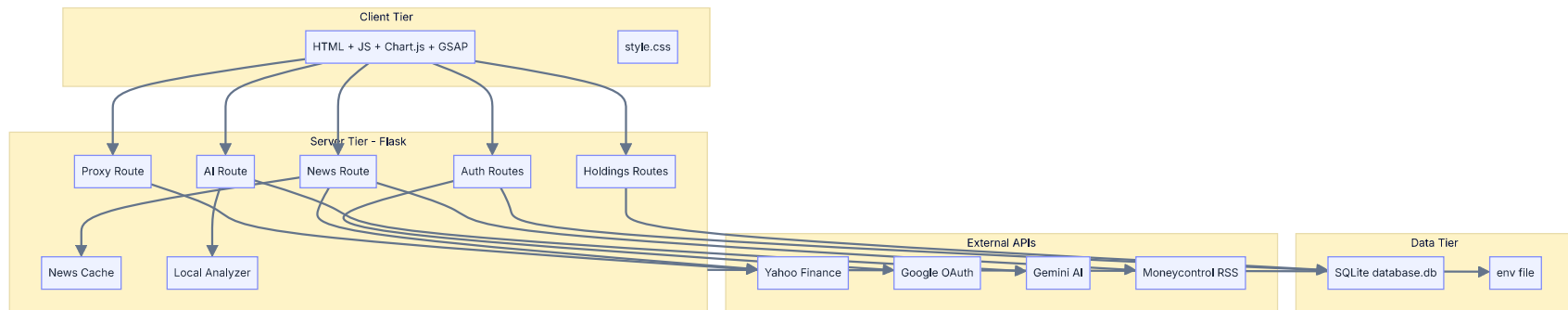




Section 6E

Component Diagram

COMPONENT DIAGRAM -- THREE-TIER ARCHITECTURE

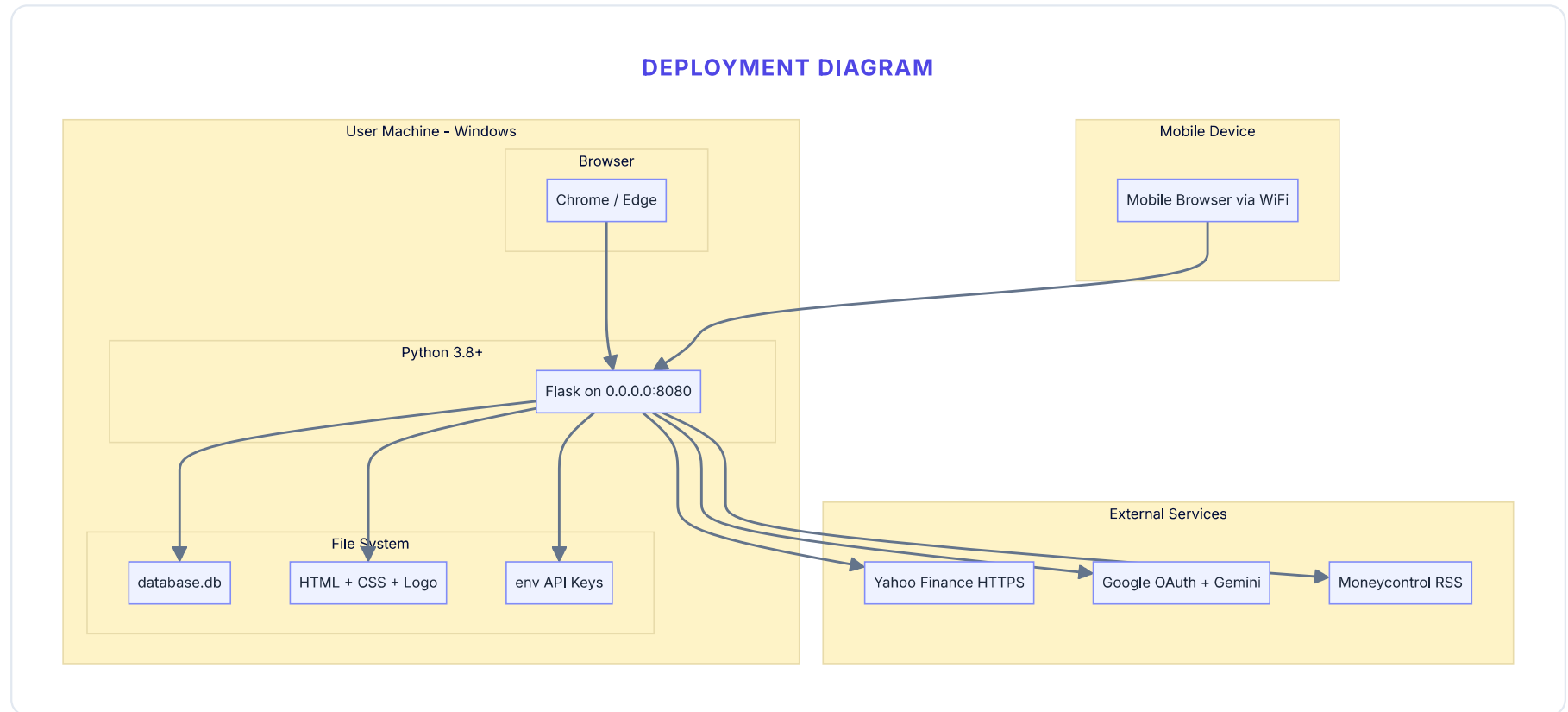


Component Responsibilities

COMPONENT	LINES	PURPOSE	WHY IT EXISTS
app.py	1,085	All server logic	Single entry point eliminates import chains
HTML	~3,300	Complete frontend SPA	Zero page reloads, all client-side rendering
style.css	~3,900	Design system	Separated for independent styling updates
Proxy Route	~30	CORS bypass	Critical: browsers block Yahoo requests without it
News Cache	~10	10-min TTL	Prevents hammering Moneycontrol servers
Local Analyzer	~130	Fallback AI	Ask AI works even without API keys

Section 6F

Deployment Diagram



Deployment Details

ASPECT	CONFIGURATION	WHY
Host: 0.0.0.0	All interfaces	Allows localhost + LAN access (mobile over WiFi)
Port: 8080	Non-privileged	Port 80 requires admin. 8080 works without elevation.
Debug: True	Development mode	Detailed errors. Must be False in production.
No Nginx/Docker	Direct Flask	Unnecessary complexity for a local mini project.
Mobile Access	Same WiFi LAN IP	Server prints IP on startup for easy access.

Section 7

API Endpoint Reference

METHOD	ENDPOINT	AUTH	PURPOSE
GET	/	No	Serve frontend SPA
GET	/api/session	No	Check login status
POST	/api/register	No	Create user account
POST	/api/login	No	Authenticate user
POST	/api/logout	Yes	Destroy session
GET	/api/login/google	No	Start OAuth flow
GET	/api/login/google/callback	No	Complete OAuth
GET	/api/holdings	Yes	Get user holdings
POST	/api/holdings	Yes	Save all holdings
GET	/proxy/url	No	CORS bypass for Yahoo
POST	/api/ask-ai	Yes	AI portfolio analysis
GET	/api/news	Yes	Portfolio news feed
GET	/admin	Admin key	Admin dashboard

Section 8

File Structure

```
d:\NEW PROJECT\mini project\  
  app.py                # Flask backend (1,085 lines)  
  PORTFOLIO TRACKER.html # Frontend SPA (~3,300 lines)  
  style.css             # Design system (~3,900 lines)  
  database.db           # SQLite (auto-created)  
  env                   # API keys (GOOGLE, GEMINI)  
  requirements.txt       # Python deps (flask)  
  logo.png              # Brand logo (512x512)  
  logo_icon.png         # Mountain icon (200x200)  
  .gitignore            # Git ignore rules
```

Important: The database.db file is auto-generated on first run via init_db(). The env file contains sensitive API keys and must never be committed to version control.